



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

ADICLEY DE OLIVEIRA COSTA - 202100104262
ALÍRIA DE SANTANA DE AMORIM CRUZ - 201900050813
HERBERT BARRETO FREIRE - 202200123977
IAGO SOARES DE MACEDO - 202100045644
LETÍCIA CAROLINE DA SILVA OLIVEIRA - 202000047655
LUCAS ARAGÃO DAMACENO - 202100045760
RODRIGO NUNES DE SANTANA - 202000047780

EVOLUÇÃO DE SOFTWARE
ATIVIDADE 3 (Parte 2) - Otimizando a Esteira de Automação

Os artefatos necessários para a realização desta atividade, bem como o vídeo tutorial dela, estão disponíveis no seguinte repositório do GitHub: [Evolucao_Software_2025-2_eanything-llm_atividade_3_parte_2](#).

Link para o fork do projeto: [fork_anything_llm](#)

Link do vídeo: https://youtu.be/_U9m7BQqIP0

Contribuições

Aluno	Matrícula	Contribuição
Adicley de Oliveira Costa	202100104262	Introdução e Linter/Formatação
Alíria de Santana de Amorim Cruz	201900050813	Antes x Depois do tempo de execução
Herbert Barreto Freire	202200123977	Documentação de CI
Iago Soares de Macedo	202100045644	Tutorial: Como Rodar Testes e Pipeline Localmente
Letícia Caroline da Silva Oliveira	202000047655	Introdução e Linter/Formatação
Lucas Aragão Damaceno	202100045760	Breve Reflexão
Rodrigo Nunes de Santana	202000047780	Tutorial: Como Rodar Testes e Pipeline Localmente
Leonardo Lima Araujo	201900125781	Não entrou em contato / não contribuiu

Cenário A (O projeto JÁ tinha CI/CD) - "Otimização"

Objetivo: Implementar duas melhorias técnicas no pipeline existente.

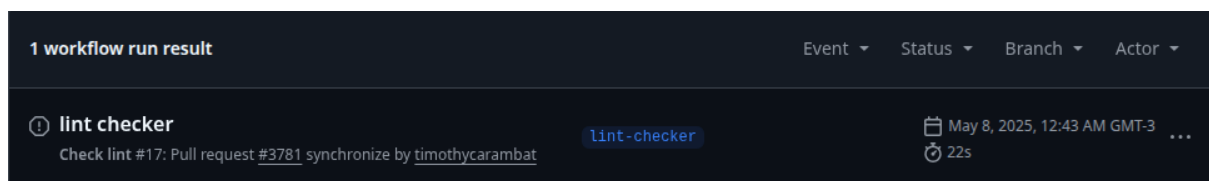
1 - Introdução e escolha das melhorias para o pipeline existente:

A realização de uma intervenção técnica concreta na infraestrutura de CI/CD é fundamental para tornar o processo de desenvolvimento de projetos open-source mais acessível e confiável. Ao reduzir barreiras de entrada, novos colaboradores conseguem configurar, executar e contribuir com o projeto de forma mais rápida e segura. Além disso, melhorias na automação aumentam a confiabilidade do build e dos testes, diminuindo falhas inesperadas e retrabalho. Esse fortalecimento da esteira de integração contínua promove feedback mais rápido e maior estabilidade no projeto. Assim, pequenas otimizações técnicas contribuem diretamente para a qualidade, colaboração e evolução sustentável do software.

1.1 Linter/Formatação:

Ao analisar o código-fonte, notamos que o projeto já implementa algumas das configurações listadas na parte 2 da atividade, como, por exemplo, o cache de dependências. Além de não possuir testes instáveis nos logs do workflow.

No entanto, ao realizar a busca das execuções do workflow "Check lint", notamos que o projeto possui apenas um arquivo lint (check-lint.yaml) o qual se encontra na branch "ab4708b", que possui apenas um registro de execução realizado: 08/05/2025 (9 meses atrás), como mostrado na figura.



Nesse sentido, decidimos criar um arquivo de configuração "super-linter.yaml" utilizando o super-linter (v8.4.0) do Github Actions para identificar possíveis problemas de formatação nos arquivos dado a não presença dessa config nas demais branches do projeto.

Durante a execução do super-linter, foi identificado um volume significativo de problemas de qualidade de código, distribuídos entre erros de formatação e estruturais. O módulo BIOME_LINT sozinho reportou 4.860 erros, 3.736 warnings e 777 infos, indicando uma baixa padronização do código no estado atual do repositório.

Além disso, diversos linters específicos (ESLint, Prettier, YAML, HTML, JSON e Markdown) também reportaram falhas, reforçando que os problemas de um projeto não se concentram apenas na lógica do código, mas também em estilo, configuração e consistência estrutural.

Através da análise dos logs, os problemas recorrentes e os seus impactos no projeto foram agrupados da seguinte forma:

Categoria do problema	Impacto no projeto
Variáveis não utilizadas	Poluição de código e baixa manutenção
Uso incorreto de Object.prototype	Possível erro em runtime
Variáveis indefinidas	Pode gerar falha em execução
Blocos vazios	Código incompleto ou lógica ausente
Conversão booleana redundante	Código menos legível
Atribuição dentro de condição	Possível bug lógico
Caracteres inválidos em regex	Pode quebrar parsing
Declarações inválidas em switch/case	Problema de escopo
Formatação inconsistente	Falha de padronização
Uso inseguro de optional chaining (?.)	Problema de robustez
Props não validadas (React)	Risco de erro em UI
Import Node incorreto	Padrão moderno não seguido

A grande quantidade de erros encontrada indica que a configuração de lint implementada no pipeline está funcionando corretamente como ferramenta de detecção de problemas, e mostrou que o projeto apresenta deficiências como a baixa padronização de código, alto acúmulo de dívida técnica e ausência de correções automáticas configuradas. Sendo assim, a adoção de correções automáticas e o tratamento gradual dos erros na base de código tendem a aumentar significativamente a confiabilidade, reduzir futuras falhas e facilitar a evolução do projeto.

1.2 Documentação de CI:

Como a análise da Parte 1 classificou a barreira de entrada do projeto como baixa a moderada – devido ao pipeline de CI/CD já robusto e automatizado –, não foi necessária a criação ou melhoria de seções no README.md ou CONTRIBUTING.md. O projeto já possui um CONTRIBUTING.md abrangente e um fluxo de integração contínua que fornece feedback rápido e seguro para novos contribuidores, atendendo adequadamente às necessidades de onboarding sem exigir intervenção documental adicional.

1.3 Tutorial: Como Rodar Testes e Pipeline Localmente

Este tutorial mostra como executar a suíte de testes e simular o pipeline de CI/CD do Anything-LLM localmente, permitindo validar alterações antes de enviar um Pull Request. Todos os comandos devem ser executados no terminal, a partir da raiz do projeto.

1.3.1 Pré-requisitos

- Node.js 18+
- Yarn 1.x
- Docker e Docker Compose
- Conta no GitHub (para fork/clone)

1.3.2 Configuração do Ambiente

- git clone <https://github.com/adicley/anything-llm.git>
- cd anything-llm
- yarn install
 - cd server && yarn install
 - cd ../frontend && yarn install
 - cd ../collector && yarn install
 - cd ..
- cd docker && cp .env.example .env
- docker-compose up -d postgres (para versões mais novas: docker compose up -d postgres)
- yarn prisma:setup

1.3.3 Executando Testes

- Testes unitários: yarn test
- Verificação de lint: yarn lint
- Verificação de formatação: yarn format:check
- Formatação automática: yarn format

1.3.4 Simulando o Pipeline com act

1. Instalação do act:

```
brew install act # macOS
```

2. Executar workflow de testes:

```
act pull_request -W .github/workflows/run-tests.yaml
```

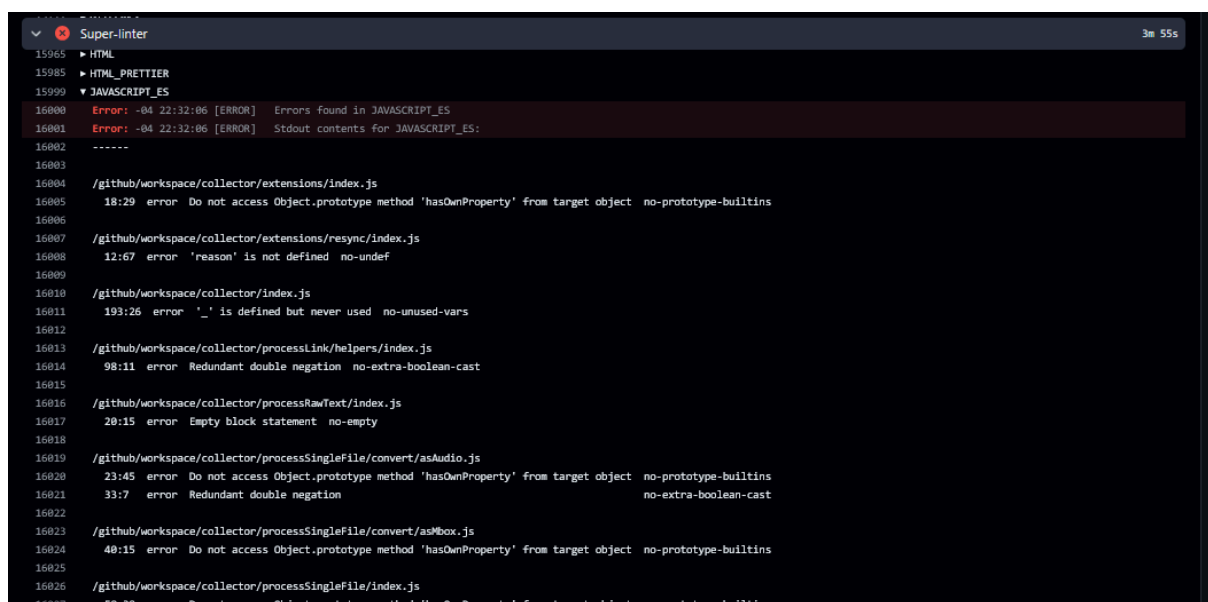
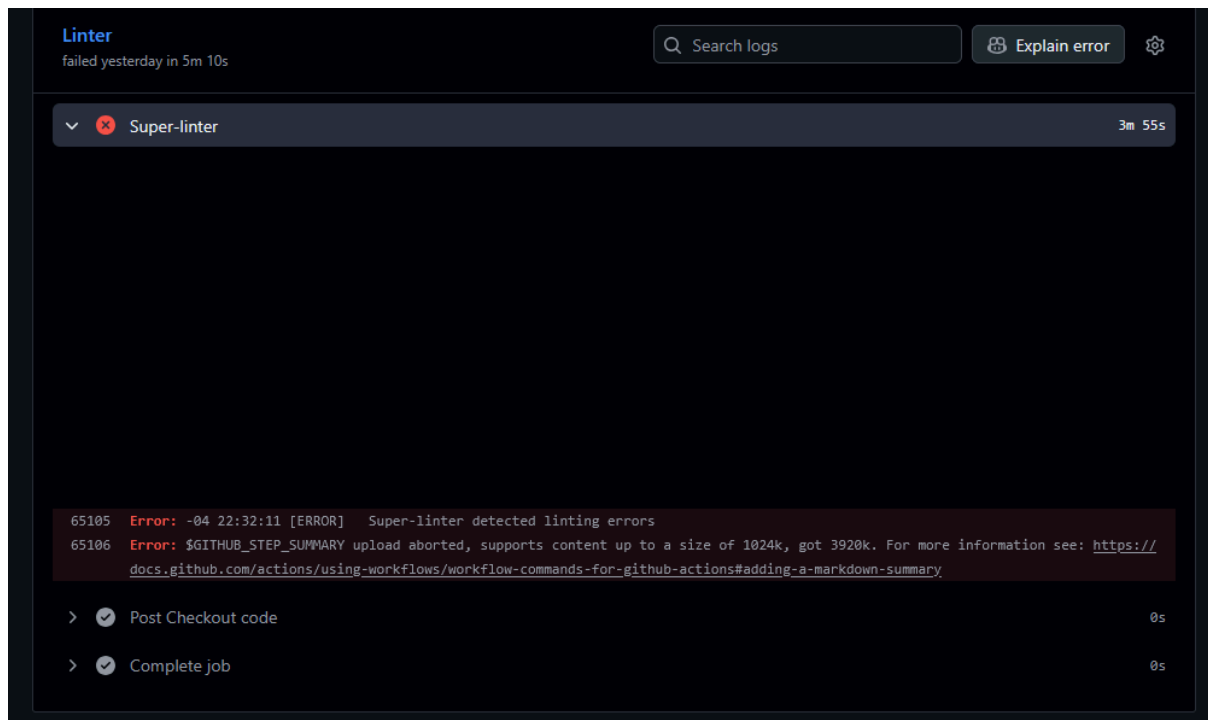
3. Executar workflow de lint:

```
act pull_request -W .github/workflows/check-lint.yaml
```

2 - Link do Pull Request (PR) ou Commit:

<https://github.com/adicley/anything-llm/actions/runs/21690816474>

3 - "Antes x Depois" (Máx. 1 página PDF): Cenário A: Print do tempo de execução anterior vs. atual (para melhoria de performance) ou Print do erro capturado pelo novo Linter.



Conforme apontado na seção 1.1, o novo job do super-linter agiu como uma "gate de qualidade", interrompendo o pipeline imediatamente após a identificação da falha de padronização.

4 - Breve Reflexão:

A implementação de ferramentas de análise estática, como o Super-Linter, impacta diretamente a "Coragem em Refatorar" ao substituir a incerteza pela validação automatizada. Ao expor explicitamente milhares de inconsistências e dívidas técnicas que antes estavam ocultas, o pipeline oferece ao desenvolvedor uma rede de segurança: a garantia de que melhorias na estrutura do código não introduzirão erros triviais de formatação ou lógica básica. Essa estabilidade na esteira de CI/CD reduz o medo de modificações em larga escala, promovendo uma cultura de evolução contínua onde o foco pode estar na melhoria da arquitetura, sabendo que a integridade do sistema é monitorada a cada commit.

Referências:

[1] AnythingLLM. Disponível em:

<https://github.com/Mintplex-Labs/anything-llm>

[2] Super-Linter Doc. Disponível em:

<https://github.com/marketplace/actions/super-linter>

[3] LLM generativa auxiliar: Gemini 3 (Seção 3)